Jupyter Notebook Demonstration: Machine Learning in Python

The following presentation serves as a walk-through and detailed explanation of the provided Example Jupyter Notebook covering machine learning in Python.

This Notebook loads pre-generated datasets provided by the libraries used to perform the machine learning tasks and performs those tasks.

Here, we will step through the example code and explain each block's function.

Part 1: K Means

Code:

```
import numpy
import scipy
import sklearn
from sklearn.datasets import load_wine
from scipy.cluster.vq import kmeans2, whiten
```

Explanation: The above sequence of statements import the necessary libraries for clustering using K Means. The numpy library is not used directly, but is used by the other libraries for better handling of matrices than base Python. The ***load_wine*** function contains a dataset of wine and its features that will be used to demonstrate machine learning techniques. The ***kmeans2*** function is an implementation of K-Means from the scipy library and the ***whiten*** function is a data normalization function that is necessary for K-Means to be effective.

Code:

```
[data, target] = load_wine(return_X_y=True)
whitened = whiten(data)
```

Explanation: The first line of code loads the wine dataset. The parameter passed to the load_wine function returns the dataset as two separate arrays, a data array and a target

array. The data array contains the features from the load_wine dataset and the target array contains the class labels. The second line of code normalizes the values of each feature in the data matrix to a feature of unit length. Whitening the data is needed since K Means considers all features numerically equal, so a feature with a range of 1000-9000 will be given greater than one with a 1-9 range without whitening.

Code:

```
start = [whitened[0],whitened[int(len(whitened)/2)], whitened[len(whitened)-1]]
[centroid, label] = kmeans2(whitened, start)
```

Explanation: The first line determines the starting centroids of the three clusters. Without defining starting centroids, the K Means algorithm will select random centroids, which will cause unpredictable behavior and inconsistent clustering. The second line of code initializes and performs the actual K-Means algorithm. After it finishes, the centroid variable will contain the final centroids for the three clusters detected by K-Means. And the label variable will contain the final cluster assignments for each data instance.

Code:

```
errors = 0.0
for i in range(len(label)):
        if target[i] != label[i]:
                errors += 1
acc = (len(data) - errors) /len(data)
print (acc)
```

Explanation: The above lines evaluate and report the accuracy of the clustering. In this case, we simply count up the data instances that were assigned to a different cluster than their true label. This is a simple example, and more complex assessments of clustering reliability would be necessary for more complex datasets.

Part 2: Decision Trees

Code:

```
import numpy
```

```
import scipy
from sklearn.datasets import load_wine
from sklearn import tree
```

Explanation: The first lines serve the same function as in the k-means example. Here, the Decision Tree function ***'tree'*** is part of the sklearn or Scikit Learn library as opposed to k-means coming from scipy.

Code:

```
[data, target] = load_wine(return_X_y=True)
```

Explanation: Again, load the dataset as separate feature values and true class labels. Since Decision Trees consider each feature individually, it is not necessary to whiten the feature matrix.

Code:

```
data_train = data[0:len(data)-2]
target_train = target[0:len(data)-2]
```

Explanation: To demonstrate the effectiveness of Decision Tree, we create a training set using leave-one-out cross validation. In this case, the final data instance is left out of the data.

Code:

```
dtc = tree.DecisionTreeClassifier()
dtc = dtc.fit(data_train,target_train)
print (dtc.predict(data[len(data)-1].reshape(1,-1)))
```

Explanation: The first line is creating an instance of a Decision Tree Classifier. The second line is used to train a Decision Tree using a training dataset. The final line is the testing step and also reports the prediction of the Decision Tree. If we were performing leave-one-out cross validation for real, we would randomly select a large number of

samples to leave out and train a new tree for each sample left out, accumulate the results of the testing over all the samples.

Part 3: Support Vector Machines (SVM)

Code:

```
import numpy
import scipy
from sklearn.datasets import load_wine
from sklearn import svm
```

Explanation: The first three lines serve the same function as in the K-Means example. Here, the ***Support Vector Machine (svm)*** function is a part of the sklearn or Scikit Learn library as opposed to k-means coming from scipy

Code:

```
[data, target] = load_wine(return_X_y=True)
data_train = data[0:len(data)-2]
target_train = target[0:len(target)-2]
```

Explanation: Again, the dataset is loaded as features and labels and divided into a training and test set. Like Decision Trees, SVMs do not generally require data whitening.

Code:

```
clf = svm.LinearSVC()
clf.fit(data_train, target_train)
print (clf.predict(data[len(data)-1].reshape(1,-1)))
```

Explanation: The first line is creating an instance of a Linear Support Vector Classification (LinearSVC) class within the SVM Model from the sklearn library in Python. The second line is used to train a SVM Classifier using the training dataset. In the last line of code, we are making a prediction based on exactly the same data instance that we used in the Decision Tree classifier example.

In these two examples, the Decision Tree and SVM classifier both correctly predict the class of the last sample. Can you find a sample that one or both classifiers do not correctly predict?